Strangler Fig Pattern

replace it slice by slice, never all at once

Watch the 4-min video

In one line: put a routing facade in front of the legacy system and move one slice of functionality at a time to new services, each move small, verified and reversible, until the old system has nothing left to do; never a big-bang rewrite.

1 Why not rewrite?

A ten-year-old monolith; everyone wants a rewrite. A big-bang rewrite means a year or more with no new features while the new system chases a moving target, then a cutover weekend that concentrates all the risk in one day. Many rewrites are abandoned. The **strangler fig** (named by Martin Fowler after a fig that grows around a host tree until the tree dies) replaces the system while it keeps running.

2 The facade

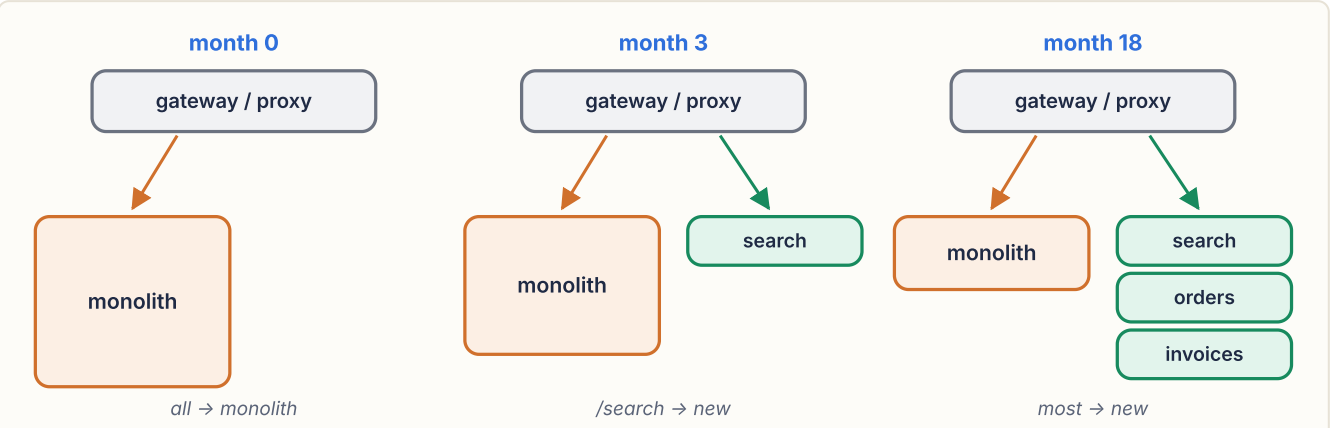


Figure 1. Clients always call the same URLs; only the routing rules behind the facade change.

Put a routing layer in front of the monolith: an API gateway, a reverse proxy such as NGINX or Envoy, or a load balancer with path rules. At first it sends everything to the monolith. Then you route one endpoint at a time to a new service. Clients never change URLs. Each move is a **routing change**, so it is reversible in seconds: if the new service misbehaves, flip the route back.

3 Choosing and verifying slices

- **Start small and loosely coupled:** a read-only endpoint like product search or catalog, not the tangled order core.
- **Ship in weeks, not months:** each slice delivers value on its own.
- **Parallel run for risky slices:** send each request to both old and new, serve the **old** answer, compare in the background and log every difference. Switch when differences stay at zero for a while.
- **Shift traffic gradually:** 1% → 10% → 50% → 100%, watching error rates and latency at each step.

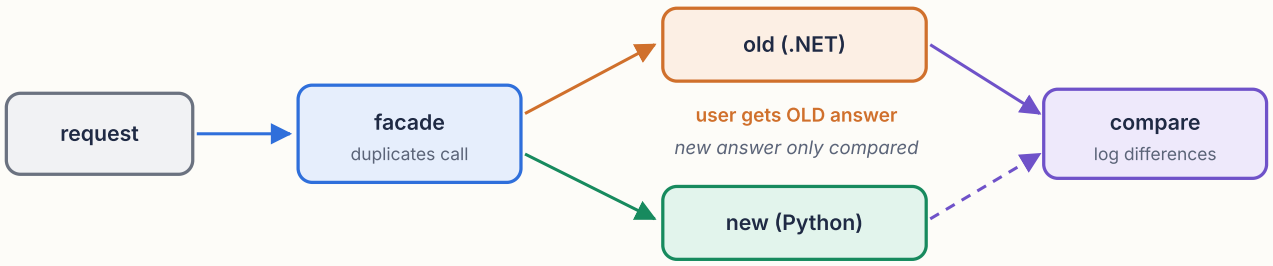


Figure 2. Parallel run ("dark launch"): users are served by the old code while the new code is checked against it.

4 Data: the hardest part

Step	Who owns the data	How the new service gets it
1. Read through	monolith	calls the monolith's API (via an anti-corruption layer, lesson 30)
2. Sync	monolith	own copy kept fresh by change data capture
3. Take over	new service	new service writes; sync back to the monolith if it still reads it
4. Clean up	new service	delete the old tables and code

Step 4 matters: if old code is never deleted, the monolith never shrinks and you run two systems for ever.

WORKED EXAMPLE: .NET MONOLITH TO PYTHON SERVICES

A gateway is put in front of a .NET monolith. First `/search` moves to `python-search-svc`, with a two-week parallel run comparing results; once differences are near zero, traffic shifts 1% → 10% → 100%. Then `/orders`, then `/invoices`. Each move is a routing change, so there is no downtime. After eighteen months only `/admin/*` still routes to the monolith.

COMMON MISTAKES

- Starting with the most tangled, most critical part.
- New services writing to the monolith's tables directly, creating a shared database.
- Never deleting migrated code, so the old system lives for ever.
- No rollback route: making a slice move irreversible.

5 Key terms

Strangler fig: incrementally replace a system behind a facade

Slice: a piece of functionality moved as one unit

Traffic shifting: move a growing percentage of requests

Facade / router: layer that decides old vs new per request

Parallel run: run old and new, serve old, compare results

Big-bang rewrite: replace everything in one cutover

6 Interview questions

Q1. What is the strangler fig pattern? BASIC

Incrementally replacing a legacy system by routing one slice of functionality at a time to new code behind a facade, until the old system can be removed.

Q2. Why not a big-bang rewrite? BASIC

It delays all value until the end, chases a moving target, and concentrates all the risk in a single cutover.

Q3. How would you migrate a large monolith with no downtime? SCENARIO

Put a routing facade in front, move one slice at a time, verify risky slices with a parallel run, shift traffic gradually, keep every step reversible, and delete old code once a slice is done.

Q4. Which slice do you move first? SCENARIO

Something small, valuable and loosely coupled, such as a read-only search or catalog endpoint, so the team learns the process with low risk.

Q5. How do you verify a rewritten pricing endpoint gives the same answers? TRICKY

Parallel run: call both, serve the old result, compare responses in the background and investigate every difference before shifting traffic.

Q6. The new service needs data the monolith owns. What now? TRICKY

Read through the monolith's API first, then keep a copy synced with CDC, then transfer ownership to the new service and remove the old tables.

Next: SD-L-012 · Anti-Corruption Layer backendroadmap.com